Slicing

Cuts between the boxes, left-out bounds, steps and negative steps, and how many items a slice holds.

THE QUESTION

How do you take part of a list or a string: the first three items, every second one, or all of it backwards?

THE INTUITION

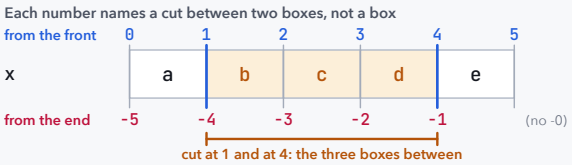
Number the cuts between the items, not the items. `x[start:stop]` cuts in two places and keeps what lies between, so the item at `stop` is left out. A third number, the step, sets the jump.

KEY TAKEAWAY

`x[start:stop:step]` begins at `start`, jumps by `step` and stops before `stop`. It never fails on bounds out of range, and it builds a new list.

`x[start:stop:step]` → start: first index taken → step: the jump → stop: first index not taken → a new list

1 Numbers name the cuts, not the boxes



`x[1:4]` → `['b', 'c', 'd']`
`x[-4:-1]` → `['b', 'c', 'd']`
`x[-4:3]` → `['b', 'c']`
`x[1]` → `'b'`

A **slice** `x[start:stop]` takes a run of items from a list or a string. Cut 0 is before the first box and cut 5 after the last; the slice keeps the boxes between cut `start` and cut `stop`, so the item at index `stop` is **left out**. Negative numbers count cuts from the end. A single index such as `x[1]` (see the lists sheet) is the box just right of its cut.

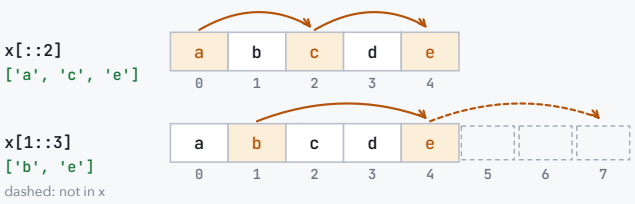
2 Leave out a bound to run to the end



no start: begin at the front (0) · no stop: go to the end (len(x))
x[:3] is short for x[slice(None, 3, None)]; None means "left out"

No `start`: begin at the front. No `stop`: run to the end. So `x[:3]` is the first three items, `x[3:]` the rest, `x[:]` all of them, and `x[:k] + x[k:]` gives back `x` for every `k` (checked for `k` from −8 to 8: `True`). A **slice object** holds the three numbers.

3 A step jumps over items



trace `i = 1: takes 'b'` `i = 4: takes 'e'` `i = 7: not below 5, stop`

The third number, the **step**, is the size of each jump: `x[::2]` takes every second item, like `x[0:5:2]`. The index grows by the step while it stays below `stop` (here the end, 5). A step of 0 never moves: `x[::0]` gives `ValueError: slice step cannot be zero`.

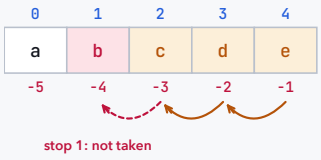
4 Count the items before you run it

slice	start	stop	step	(stop-start+step-1)//step	len()
`x[1:4]`	1	4	1	`(4-1+0)//1 = 3`	3
`x[::2]`	0	5	2	`(5-0+1)//2 = 3`	3
`x[1::3]`	1	5	3	`(5-1+2)//3 = 2`	2
`t[1:10:4]`	1	10	4	`(10-1+3)//4 = 3`	3
`t[4:50]`	4	11	1	`(11-4+0)//1 = 7`	7
`x[3:1]`	3	1	1	`(1-3+0)//1 = -2, so 0`	0

t = "programming" (11 characters). Fill in a missing bound and clip start and stop to 0..len first; then the formula matches len() every time

With step 1 a slice has `stop - start` items, or none if that is negative. With a step `k` above 0 the count is `(stop - start + k - 1) // k`, the division rounded up. The table checks this against `len()` on six slices. `x[3:1]` starts past its stop, so it is empty.

5 A negative step walks right to left



`x[::-1]` → `['e', 'd', 'c', 'b', 'a']`
`x[4:0:-1]` → `['e', 'd', 'c', 'b']`

`x[4:1:-1]` → `['e', 'd', 'c']`

i	i > 1	takes
4	True	'e'
3	True	'd'
2	True	'c'
1	False	stop

no bounds: last box to past the front
stop 0 leaves 'a' out

With a negative step the slice starts at box `start`, moves left, and stops before box `stop`, still left out. Read the numbers as box positions here. `x[::-1]` reverses the whole list. The notebook's `x[4:0:-1]` misses `'a'` because box 0 is the stop.

6 Negative steps: the edge cases, run for real

slice	the REPL prints	why
`x[5:1:-1]`	`['e', 'd', 'c']`	start 5 is past the end: moved to 4
`x[1:5:-1]`	`[]`	walking left from 1 never reaches 5
`x[4:-1:-1]`	`[]`	-1 is box 4, so it stops at once
`x[4::-1]`	`['e', 'd', 'c', 'b', 'a']`	no stop: all the way past the front
`x[1::-1]`	`['b', 'a']`	the first two, reversed
`x[:-3:-1]`	`['e', 'd']`	the last two, reversed
`x[-3::-1]`	`['c', 'b', 'a']`	all but the last two, reversed

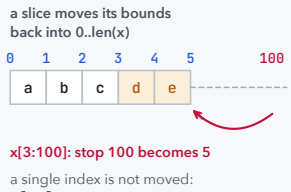
A start past the end moves back to the last box; a start left of `stop` gives `[]`, not an error. `-1` always means the last box, never "before the front", so `x[4:-1:-1]` is empty: leave the stop out instead. The last three rows are the notebook's reversed recipes.

Slices are safe, new and assignable

Out-of-range bounds, slicing strings, palindromes, why a slice is a copy, and replacing a range.

7 Out-of-range slices never raise

```
>>> x[10]
IndexError: list index out of range
>>> x[0:10]
['a', 'b', 'c', 'd', 'e']
>>> x[3:100]
['d', 'e']
>>> x[10:20]
[]
>>> x[-100:2]
['a', 'b']
>>> x[4:2]
[]
```



An index outside the list raises an **IndexError** (see the lists sheet). A slice never does: Python moves `start` and `stop` back into 0 to `len`, so `x[3:100]` is the last two items and `x[10:20]` is empty. Handy for "the first 3, if there are 3", but a wrong bound gives a wrong answer with no message.

8 Strings slice the same way



`s[:6]` → `'Python'` `s[::2]` → `'Pto lcs'`
`s[7:]` → `'slices'` `s[::-1]` → `'secils nohtyP'`

w	w[::-1]	w == w[::-1]
level	'level'	True
racecar	'racecar'	True
python	'nohtyp'	False

w	w[::-1]	w == w[::-1]
noon	'noon'	True
Level	'leveL'	False
12321	'12321'	True

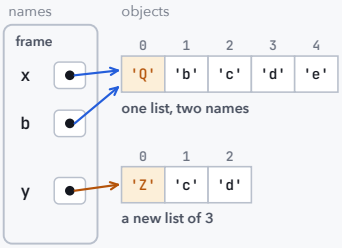
A string is a sequence of characters, so the same rules apply, and a slice of a string is a string (`<class 'str'>`). A **palindrome** reads the same both ways: `w == w[::-1]` tests it; capitals count. `str(n)[::-1]` reverses the digits of a number in one step (the while loops sheet used a loop).

9 A slice is a new list

copy.py

```
1 x = ['a', 'b', 'c', 'd', 'e']
2 y = x[1:4]
3 y[0] = 'Z'
4 b = x
5 b[0] = 'Q'
6 print(x)
7 print(y)
```

line 6: `['Q', 'b', 'c', 'd', 'e']`
line 7: `['Z', 'c', 'd']`



Slicing builds a **new list** and copies the items in. `y = x[1:4]` gets its own list, so `y[0] = 'Z'` leaves x alone. `b = x` copies nothing: `b` is a second name for the same list, so `b[0] = 'Q'` shows in x. `c = x[:]` is a full copy: `print(c == x, c is x)` gives `True False`. Why: the names and objects sheet.

10 Assigning to a slice edits the list

statement	takes out	x afterwards	len
`x[1:3] = 'B', 'C'`	`'b', 'c'`	a B C d e	5→5
`x[1:4] = 'Z'`	`'b', 'c', 'd'`	a Z e	5→3
`x[1:2] = 'P', 'Q', 'R'`	`'b'`	a P Q R c d e	5→7
`x[2:2] = 'N', 'M'`	`[]`	a b N M c d e	5→7
`x[1:4] = []`	`'b', 'c', 'd'`	a e	5→2

each row starts again from x = ['a', 'b', 'c', 'd', 'e'] (len 5) · green: new items

Slice assignment `x[i:j] = new` swaps the items between cut `i` and cut `j` for the items of `new`; the counts may differ, so the list can shrink or grow. Assigning to the empty `x[2:2]` inserts; assigning `[]` deletes, like `del x[1:4]`. With a step the counts must match, or you get a **ValueError**.

11 Common mistakes: what you wrote, what Python says, the fix

you wrote	the last line Python prints, or what you see; why, and the fix
`x[1:3]`	`prints: ['b', 'c']` the stop is left out; `x[1:4]` prints `['b', 'c', 'd']`
`x[4:0:-1]`	`prints: ['e', 'd', 'c', 'b']` box 0 is the stop, so `'a'` is left out; `x[::-1]` prints `['e', 'd', 'c', 'b', 'a']`
`x[::0]`	`ValueError: slice step cannot be zero` a step must move; use 1, or −1 to go backwards
`w[0] = "J"`	`TypeError: 'str' object does not support item assignment` a string cannot be changed; build a new one: `w = "J" + w[1:]` gives `Jello`
`x[1:2] = 'hi'`	`prints: ['a', 'h', 'i', 'c', 'd', 'e']` a string is split into its characters; wrap it in a list: `['hi']` gives `['a', 'hi', 'c', 'd', 'e']`

12 Try it (answers in green)

1. `"banana"[1:4]`?
ana: boxes 1, 2 and 3.

2. `[3, 8, 1, 9, 4][::-2]`?
`[4, 1, 3]`: from the end.

3. `len("banana"[2:100])`?
4: the stop is clipped to 6.

4. `"stressed"[::-1]`?
desserts.

5. `a = [1, 2, 3, 4, 5]`, then `a[1:4] = [0]`. Now a?
`[1, 0, 5]`: three out, one in.

6. `w = "python"; w[-3:] + w[:-3]`?
honpyt: a rotation by 3.

start in, stop out · no bound: to the end · a negative step walks back · bounds are clipped · a slice is new

Source: the slicing notebook and its picture of positive and negative indices, nipunbatra.github.io/comp22. Every output and error message comes from running the code with Python 3.11.11. Next: the for loops and range sheet.